SplitMend for constraint aware repair of machine learning dataset splits in Python

Ravi Shanker Thadishetti

University of the Cumberlands, United States

Corresponding author: thadishetti.ravishanker@gmail.com

Abstract

Existing machine-learning partitions can require revision when researchers introduce validation sets, discover repeated entities, or change temporal evaluation requirements. Rebuilding a split may unnecessarily move previously assigned observations. SplitMend is an MIT-licensed Python library that minimizes a weighted assignment-change objective subject to explicit group separation, locked memberships, temporal eligibility, size and class constraints. It combines integer optimization, exact aggregation of equivalent records, independent output verification and bounded conflict explanations. Across 30 workflow configurations on four public datasets, every SplitMend repair satisfied its policy and was certified optimal by the solver. In a clinical cohort of 101,766 encounters, repair reduced aggregate assignment changes by 23.47% versus violation-first greedy, but only 0.008% versus cost-aware greedy with local exchanges. The latter comparison comprised four wins and eleven ties. On human activity data, the aggregate reduction versus cost-aware greedy was 1.66%; simpler rules tied the optimum on education and temporal workflows. These results support verified repair with objective bounds, while showing that efficient heuristics can be competitive. Source data, manifests, raw outputs, tests and reproduction scripts accompany the software.

Keywords: dataset partitioning; data leakage; constrained optimization; reproducible research; Python software

Code metadata

Item	Description
Current code version	0.1.1
Permanent repository or archive	Accompanying SplitMend research archive; public release URL and DOI not yet assigned
Legal code license	MIT; original datasets retain CC BY 4.0 licenses
Code versioning system	Git
Software languages	Python 3.11 or later
Requirements	NumPy 1.26 to below 3; SciPy 1.11 to below 2
Documentation	README.md, docs/API.md, docs/RELEASE.md in the source archive
Support contact	thadishetti.ravishanker@gmail.com

1 Motivation and significance

Dataset partitioning encodes the prediction task. Separating people is appropriate when evaluating generalization to unseen people, whereas forecasting requires respecting feature-history and target times. The same overlap can therefore be acceptable for one task and invalid for another. Researchers must state these assumptions rather than infer universal leakage from similarity alone. Public datasets provide realistic observations on which to study the operational consequences of these choices [1-3].

Established tools provide important parts of this workflow. Scikit-learn supplies group-aware splitting [4]. DataSAIL constructs identity- or similarity-aware partitions using optimization [5]. DataEval Flow audits existing partitions and records decisions [6], while dataset-splitter combines grouping, stratification, temporal splitting and reports [7]. SplitMend addresses an adjacent operation: revising an existing

assignment while preserving explicit locks and minimizing disruption. It does not introduce a new integer-programming algorithm or claim that auditing, grouping or provenance is individually novel.

The software contribution is a common representation connecting repair, verification and conflict explanation. A researcher can express interacting requirements, obtain a revised row-to-partition map, and inspect an independently checked result with an objective bound. Exact aggregation reduces redundant assignment variables without altering the feasible repair problem. The evaluation compares this workflow with established split constructors and repair heuristics, including a cost-aware local method and cases where optimization gives little or no practical advantage.

2 Software description

The installable src-layout package exposes Sample, Policy, audit, repair and interval_allowed, together with a JSON command-line interface. The runtime requires NumPy and SciPy and uses the HiGHS optimizer exposed through scipy.optimize.milp [8]. No external service, GPU or language-model API is required. Data acquisition, classification experiments and external comparators are optional scripts with separate pinned environments.

2.1 Input model and objective

Each sample has a unique identifier, an original partition, zero or more namespaced group tokens, a class label, eligible partitions, a lock flag and a positive integer weight. Retained members of a group must occupy at most one ordinary partition. Duplicate hashes can be supplied as group tokens; the package does not discover approximate duplicates. Eligibility constrains where a complete source/target dependency interval can be assigned. Size and class bounds are inclusive absolute counts. Quarantine is disabled unless explicitly enabled.

The objective sums row costs: zero for unchanged membership, the row weight for reassignment to another ordinary partition, and the weight multiplied by a declared penalty for newly quarantining a record. Thus the objective is a weighted trade-off, not a lexicographic promise to minimize exclusions first. The benchmark uses unit weights and a quarantine penalty of ten. Locks, eligibility and count requirements remain hard constraints.

Conceptually, binary assignment variables select one destination per row. Group-partition variables restrict retained group members to one partition. Quarantined rows do not force two otherwise distinct groups to remain connected. The implementation aggregates records only when their active groups, original assignment, label, eligibility, lock and weight are identical. Integer block counts can then be expanded to individual assignments with unchanged constraints and cost. Conversely, summing any feasible row assignment produces a feasible block assignment. This gives exact aggregation relative to the declared model; singleton group tokens can be omitted safely.

2.2 Verification and failure handling

A separate row-level verifier rechecks all memberships, locks, groups, eligibility and count bounds. The solver returns optimal, feasible, infeasible or unknown. Optimality is relative to solver numerical tolerances; time-limited feasible incumbents are not presented as optima. An unchanged valid split has the global lower bound zero. A canonical input hash identifies row metadata and policy; separate archive hashes identify the original feature data.

Direct lock/group and lock/eligibility contradictions produce small witnesses. Other infeasible cases use deletion checks over named constraint families. Only completed checks justify an irreducible conflict set, which is not claimed to have minimum cardinality. The solver budget and explanation budgets are separate. JSON outputs record the status, objective, bound, violations, assignments and runtime, and the CLI refuses to overwrite the input problem.

3 Illustrative example

Suppose records a and b belong to the same person, a is in training, and b is locked in testing. Record c belongs to another person. With group separation required, the minimum unit-cost repair moves a into

testing and leaves b and c fixed. If both a and b are locked into their original different partitions, the result is infeasible and identifies the two locks and their shared group.

```
from splitmend import Sample, Policy, repair
rows = [Sample('a', 'train', ('person:7',)),
        Sample('b', 'test', ('person:7',), locked=True),
        Sample('c', 'train', ('person:8',))]
result = repair(rows, Policy(partitions=('train', 'test')))
assert result['objective'] == 1
assert result['after']['valid']
```

The equivalent CLI call is splitmend repair examples/problem.json --output repaired.json. The output is a new assignment manifest; modifying feature files or retraining a model remains an explicit downstream action.

4 Evaluation and impact

4.1 Datasets and protocol

The evaluation comprises retrospective workflow simulations on real observations, not discoveries of defects in official dataset protocols. ChatGPT/Codex assisted implementation, experimental scripting and analysis. All numerical results were computed by executing the archived programs using the cited source datasets; no language model generated benchmark observations or performance values. HAR contributes all 10,299 feature vectors from 30 volunteers [1]. Its supplied test membership is locked; a row-stratified validation subset is initially drawn from the official training set. Repair then enforces subject separation. OULAD contributes all 32,593 student-module-presentation rows, representing 28,785 distinct student IDs [2]. Initial 60/20/20 row-stratified partitions are repaired for an unseen-student task with original test rows locked. These workflows use seeds 0-4.

For these classification workflows, partition-size bounds allow an additive five percentage points around original proportions. Per-class bounds allow eight percentage points with at least one row of each class in every partition. The policies are supplied to all audits. Beijing contributes 420,768 hourly observations from 12 stations [3]. Twenty-four source hours and a target 24 hours ahead define 420,204 complete temporal spans. Five chronological boundary configurations test full-span containment. Boundary-crossing spans require explicit quarantine; missing pollutant values are not imputed because this experiment audits temporal dependencies rather than fitting a forecaster.

Comparators are weighted group-majority repair, a greedy count-violation repair, scikit-learn GroupShuffleSplit, DataSAIL 1.4.0 I1 and dataset-splitter 0.1.0. External-tool adapters first assign groups containing locked test records to testing and split the remaining groups. DataSAIL is called through its pinned I1 solver to supply mixed-class group count vectors. Its relative lower-bound tolerances are conservatively mapped to the common absolute bounds. Native and tighter fixed-tolerance runs are also retained. This mapping was added during implementation after inspecting policy mismatches, and is not a preregistered comparison.

The clinical extension uses all 101,766 encounters from 71,518 patients in the UCI Diabetes 130-US Hospitals table [9]. It supplements the original datasets. For seeds 0-4, 20% of patients form a locked test cohort; all other encounters initially receive train/validation assignments at 75/25 within joint readmission, age and gender strata. Four age bands (under 40, 40-59, 60-79, 80+) produce 25 nonempty joint strata with the published outcome and gender categories. Unknown gender values are retained. Patient separation, no quarantine and train/validation eligibility are mandatory. Size and stratum bounds allow 1, 2.5 or 5 percentage points around the target partition fraction, using floor/ceiling bounds on each available count. All 15 configurations are reported. The table has no hospital identifier or admission timestamp; this is neither site-held-out nor chronological evaluation.

The clinical protocol and comparator parameters were committed before clinical comparisons, after observing the original study; this is an exploratory extension, not external preregistration. Comparators include patient majority, group re-splitting, violation-first greedy and cost-aware greedy with feasible local exchanges. The cost-aware rule selects added movement cost per unit violation reduction, then improves feasible solutions using single-patient flips and pairs from up to 256 lowest-cost profiles in each direction. Both greedy methods and SplitMend have 60-second search/optimization budgets, with preprocessing and

verification measured separately. After observing clinical near-ties, the unchanged cost-aware method was also applied to all HAR/OULAD cases. That adapter fixes groups forced into test and searches remaining train/validation memberships; it does not explore unrelated groups moving into test. Both protocol stages and all outcomes are archived.

All outputs pass through the same row-level audit. Invalid results remain in the raw archive and are excluded from successful-repair cost summaries. Split constructors do not optimize edit cost, so differences from them demonstrate the consequences of the repair objective rather than a superior solution to their original problem. Primary evidence is validity and assignment cost; timings are descriptive single-host measurements. Fifteen zero-cost fixed-point controls, three deliberately contradictory lock controls, 100 exhaustive problems of two to six rows, and compressed/uncompressed comparisons supplement the real-data cases.

4.2 Repair results

SplitMend produced verified, solver-certified optima for all 30 configurations. Table 1 reports HAR and OULAD; each listed method satisfied the common policy in all five runs. HAR required a mean of 2,009.8 changes. Relative to cost-aware greedy, the aggregate saving was 170 changes (1.66%), comprising one win and four ties. The original simpler greedy comparison gave 4.92% fewer changes. OULAD required 2,140.6 changes; majority and both greedy repairs already attained this optimum. Its 80.03% reduction versus group re-splitting measures preservation of earlier assignments, not superiority for constructing fresh splits.

Table 1. Mean assignment changes over five valid runs. All methods preserve the original locked test rows. DataSAIL and dataset-splitter include the documented adapters.

Method	HAR changes	OULAD changes
SplitMend	2,009.8	2,140.6
Original greedy repair	2,113.8	2,140.6
Cost-aware greedy + exchanges	2,043.8	2,140.6
Group re-split	2,478.8	10,717.8
DataSAIL I1	2,482.2	10,909.2
dataset-splitter	2,399.6	10,760.8

Every Beijing configuration quarantined 1,128 boundary-crossing windows and retained 99.73% of input windows. The direct temporal baseline gave exactly the same result. Majority repair was invalid on HAR because it eliminated required validation coverage. Native DataSAIL tolerance configurations sometimes violated the custom count policy; the aligned adapter satisfied it in all ten classification cases. These are differences in supported constraint specifications, not reported defects in DataSAIL.

The clinical extension exposes the importance of comparator strength (Table 2). Both greedy methods and SplitMend satisfy all 15 policies. SplitMend saves 35,010 changes (23.47%) against violation-first greedy, but only nine changes (0.008%) against cost-aware greedy with exchanges: four wins, eleven ties and no losses. The largest single-case saving against the latter is five records. Patient-majority repair fails the count policy in every case, while group re-splitting is valid in four cases. Invalid outputs remain available. These results establish constraint enforcement and an optimum bound, but no substantial clinical-cohort advantage over the stronger heuristic.

Table 2. Clinical mean assignment changes over five seeds per tolerance. Each listed method is valid in all five runs at each tolerance; pp denotes percentage points.

Tolerance	SplitMend	Violation-first greedy	Cost-aware greedy + exchanges
1 pp	7,618.0	10,881.8	7,619.8
2.5 pp	7,610.2	9,833.8	7,610.2
5 pp	7,606.0	9,120.6	7,606.0

Across clinical tolerances, median verified times were 9.10-9.26 s for SplitMend and 2.52-2.90 s for cost-aware greedy. Seeds and tolerances are repeated configurations of one cohort, not independent clinical datasets. The joint strata impose a declared evaluation-coverage policy, not a fairness or clinical-representativeness guarantee. No patient-outcome or clinical prediction endpoint is evaluated.

Median verified wall times for SplitMend were 0.241 s on HAR, 4.135 s on OULAD and 6.751 s on Beijing. Verification time is separately recorded. The SplitMend call also includes input hashing and internal audits. Simpler comparators were generally faster; no runtime-speedup claim is made. All 100 exhaustive cases agreed with enumeration, all 15 clean controls remained unchanged, and all three contradictory-lock controls produced infeasibility witnesses.

Table 3. Aggregation ablation on selected records containing violations. Each compressed and row-level pair reached the identical optimal objective; reduction depends on repeated constraint profiles.

Dataset	Rows	Compressed variables	Row variables	Objective
HAR	1904	234	5730	396
OULAD	1999	8634	8778	1008
BEIJING	2000	132	8000	10000

4.3 Downstream use and limitations

Fixed RidgeClassifier pipelines with alpha=1 were rerun without tuning on test outcomes. HAR uses training-fitted standardization; OULAD uses training-fitted numeric preprocessing and one-hot encoding of enrolment attributes. Evaluation uses common original test rows, because lock-preserving repairs can pull additional related rows into testing. The mean HAR common-test accuracy was 96.21% before repair and 95.83% after SplitMend; OULAD was 44.65% and 44.64%, respectively. These results demonstrate executable integration and do not establish an accuracy improvement. The changed training and validation populations also prevent attributing every metric difference exclusively to leakage removal.

The evidence covers four datasets and simulated workflows. It does not establish independent user adoption, reduced retraining cost or broad domain generalization. Constraints depend on correctly supplied identities and dependency spans. Multiple optimal assignments may produce different model scores. Large instances can exceed solver budgets; the package distinguishes feasible incumbents from proved optima. Repairing a partition cannot erase historical test exposure or preserve official benchmark comparability after changing membership. Independent-host reproduction and external workflow studies remain future work.

5 Conclusions

SplitMend provides an installable, auditable workflow for revising existing dataset partitions under explicit constraints. Its contribution combines a weighted repair objective, exact record aggregation, lock-preserving assignments, independent verification and qualified conflict explanations. The experiments show verified optima, preservation of existing assignments relative to fresh splits, and small or absent improvements over stronger repair heuristics. The clinical extension strengthens constraint and reproducibility evidence without establishing heuristic dominance. Optimization provides objective bounds and consistent policy handling; fast near-optimal heuristics remain useful where those guarantees are unnecessary.

Data and software availability

The accompanying research archive contains version 0.1.1, original public dataset archives and attribution, SHA256 provenance, all derived policies and assignments, raw solver outputs, comparison scripts, pinned environment files and generated tables. The source distribution and wheel are included. Public repository and archival identifiers must be assigned during release; none are represented as already registered. The one-command harness reruns 180 method comparisons, independently re-audits their assignments and regenerates results. Seventy-two source tests and 49 core tests against an installed wheel were checked in separate environments on the same host. This is a development-host rerun, not independent reproduction.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI assisted technologies

OpenAI ChatGPT/Codex assisted research planning, software development, experiment orchestration, analysis and preparation of this manuscript. Numerical results were obtained by executing the supplied programs on the cited datasets. Responsibility for verifying the work, approving the final text and making the submission rests with the human author.

References

[1] Reyes-Ortiz J, Anguita D, Ghio A, Oneto L, Parra X. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository; 2013. https://doi.org/10.24432/C54S4K.

[2] Kuzilek J, Hlosta M, Zdrahal Z. Open University Learning Analytics dataset. Scientific Data 2017;4:170171. https://doi.org/10.1038/sdata.2017.171. Data: https://doi.org/10.6084/m9.figshare.5081998.v1.

[3] Chen S. Beijing Multi-Site Air Quality. UCI Machine Learning Repository; 2017. https://doi.org/10.24432/C5RK5G.

[4] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 2011;12:2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html.

[5] Joeres R, Blumenthal DB, Kalinina OV. Data splitting to avoid information leakage with DataSAIL. Nature Communications 2025. https://doi.org/10.1038/s41467-025-58606-8.

[6] DataEval Flow developers. Audit a set of splits before training. https://dataeval-flow.readthedocs.io/en/latest/notebooks/audit.html. Accessed 2 November 2026.

[7] dataset-splitter developers. dataset-splitter version 0.1.0. https://pypi.org/project/dataset-splitter/. Accessed 2 November 2026.

[8] SciPy developers. scipy.optimize.milp documentation. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html. Accessed 2 November 2026.

[9] Clore J, Cios K, DeShazo J, Strack B. Diabetes 130-US Hospitals for Years 1999-2008. UCI Machine Learning Repository; 2014. https://doi.org/10.24432/C5230J.